Gaussian Naive Bayes on the Bank Marketing Data: Effect of Training Set Size

Steven Stokes
CSCI 460 – Programming Assignment 1

October 26, 2026

1 Experiment Design

Data preparation. The Bank Marketing dataset (`bank-full.csv`) was loaded in Julia and cleaned by removing every row containing a missing value or the string "unknown". This reduced the data from 45,211 rows to 7,842 rows with 16 attributes and a binary target `y` (whether the client subscribed to a term deposit). Textual columns were coerced to `Multiclass`, the target to `OrderedFactor`, and integer-valued attributes to `Continuous`. The target was separated from the attributes *before* encoding, so it stays categorical, as the classifier and the accuracy/F1 measures require. The categorical attributes were then mapped to integer codes with MLJ's `OrdinalEncoder`, giving an all-`Continuous` feature table.

Model and metrics. I used scikit-learn's `GaussianNBClassifier` (loaded through `MLJScikitLearnInterface`) with default parameters. Performance was measured with accuracy and F1 score. The F1 score treats the second level of the target ("yes") as the positive class.

Task II (single split). A single 70/30 stratified, shuffled holdout split was used to train and test the model.

Task III (training-size experiment). For each training fraction $f \in \{10\%, 20\%, \ldots, 90\%\}$, I ran 10 independent iterations. Each iteration draws a new stratified, shuffled split (seed $= 100f + i$, where f is in percent and $i = 1, \ldots, 10$), trains on f of the data, and tests on the remaining $1 - f$. This gives 90 model fits in total. The mean and standard deviation of accuracy and F1 over the 10 iterations are reported for each fraction.

2 Results

The single 70/30 split in Task II gave an accuracy of 0.8003 and an F1 score of 0.6030. (This split was not seeded, so these numbers change slightly between runs.) Table 1 and Figure 1 summarize Task III. The last column of the table are approximate and are used in the discussion: n_{test} is the size of the test set.

Table 1: Performance versus training fraction (mean and standard deviation over 10 iterations).

Train	Acc. mean	Acc. std	F1 mean	F1 std	$\approx n_{\text{test}}$
10%	0.7985	0.0177	0.6033	0.0201	7058
20%	0.7993	0.0102	0.6003	0.0100	6274
30%	0.8071	0.0061	0.6047	0.0139	5489
40%	0.8064	0.0075	0.6074	0.0100	4705
50%	0.8032	0.0064	0.6004	0.0135	3921
60%	0.8053	0.0090	0.6060	0.0067	3137
70%	0.8054	0.0038	0.6089	0.0145	2353
80%	0.8089	0.0105	0.6112	0.0205	1568
90%	0.8059	0.0150	0.6095	0.0287	784

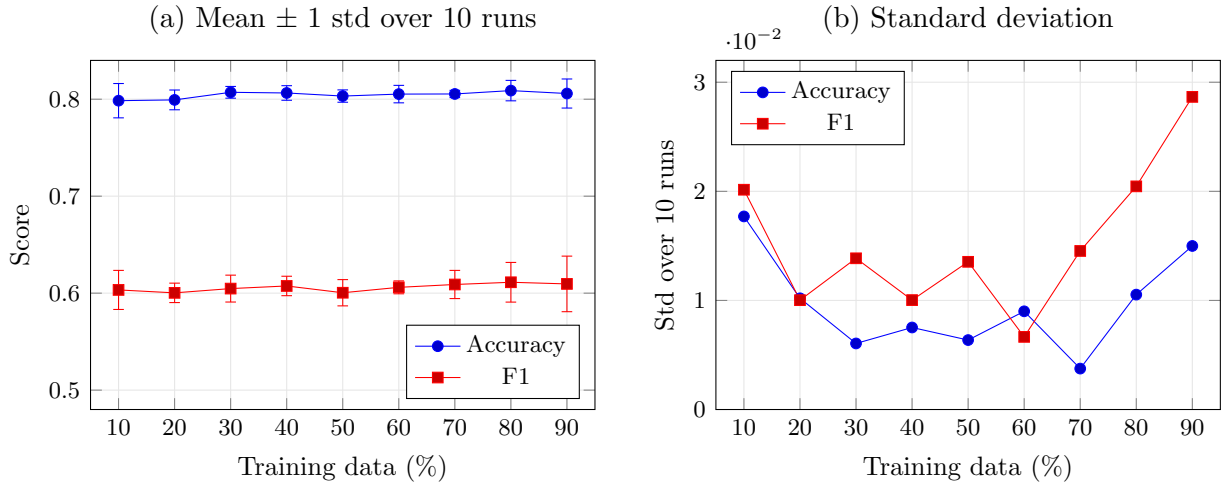


Figure 1: (a) Mean accuracy and F1 versus training fraction, with error bars of one standard deviation. (b) The standard deviations alone.

3 Discussion

Effect of training size on the mean. Increasing the training data barely changes performance. Mean accuracy moves from 0.798 at 10% to roughly 0.806–0.809 at 30–90%, and mean F1 stays between 0.600 and 0.611. Even 10% of the cleaned data (about 780 rows) gives nearly the same result as 90%. This is expected for Gaussian Naive Bayes. It is a high-bias, low-variance model with very few parameters (one mean and one variance per feature per class, plus class priors), so these estimates stabilize with a small sample. After that point, extra data cannot help because the limit is the model's assumptions, not the amount of data. Those assumptions are that features are conditionally independent given the class, and that each feature is Gaussian. The second assumption is a poor fit here: the ordinal-encoded columns (`job`, `marital`, `contact`, `month`, ...) are nominal categories with arbitrary integer codes, and several numeric columns such as `balance` and `duration` are heavily skewed.

Non-monotonic behavior. The means are not strictly increasing. For example, accuracy is 0.8064 at 40% but 0.8032 at 50%, and 0.8089 at 80% but 0.8059 at 90%. These differences are well within one standard deviation, so they are noise and not evidence that more data hurts. Because the learning curve is already flat, the small ups and downs are dominated by which examples happen to land in each split.

Standard deviation. The standard deviation of accuracy is large at 10% (0.0177), falls to a minimum around 30–70% (0.004–0.009), and rises again at 80% and 90% (0.0105 and 0.0150). F1 shows the same pattern with larger values. The two ends of the curve have different causes:

- *Small training fractions* (10–20%):The variance comes from the model itself being fit on only about 780–1,570 rows, so the estimated means and variances (especially for the minority "yes" class) change noticeably from split to split.
- *Large training fractions* (80–90%): the training sets are large and heavily overlapping, so the fitted models are nearly identical. The variance here comes from the small test set (only about 1,570 and 780 rows). The observed accuracy standard deviations (0.0105 and 0.0150) match the binomial prediction (0.0101 and 0.0143) closely, which confirms that the spread reflects evaluation noise and not instability of the model.
- *Middle fractions*: The very low value at 70% (0.0038) is lower than the binomial estimate (0.0082), which is most likely luck with only 10 runs; it should not be read as 70% being a special training size.

F1 versus accuracy. F1 is much lower than accuracy (about 0.60 versus 0.80) and its standard deviation is larger at every fraction. F1 is computed only from the positive ("yes") class, which is the harder one for Naive Bayes, and it is based on fewer test examples than accuracy, so it is noisier. This is most visible at 90%, where the F1 standard deviation (0.0287) is about twice the accuracy standard deviation. The small, positive-class-only test set makes the precision and recall estimates unstable.

Consistency with Task II. The single 70/30 result (accuracy 0.8003, F1 0.6030) falls within about 1.4 standard deviations of the 70% means in Task III (0.8054 $\pm$ 0.0038 for accuracy, 0.6089 $\pm$ 0.0145 for F1). This shows that a single split gives a reasonable estimate but not a precise one, which is why repeating the split 10 times and reporting the standard deviation is useful.

Conclusion. Gaussian Naive Bayes reaches its performance plateau (about 80–81% accuracy and 0.60–0.61 F1) with a small fraction of the data. More training data mainly reduces the variance caused by the training set, while the variance seen at the largest fractions comes from shrinking test sets. Improving performance further would require changing the model or the features, for example by using one-hot encoding for nominal attributes, transforming skewed numeric features, or using a model such as `MixedNB` that handles categorical and continuous features separately.

Github Repo: https://github.com/Steven-S1020/Machine-Learning-Programs/tree/main/Naive-Bayes-Classifier